

M08 / LEARNER BOOK

Files, APIs and ingestion

Standalone Zero-to-Job-Ready Data Engineer 2026

17 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE How to use this module	3
DE08-01 Ingestion architecture: source -> landing -> staging	5
DE08-02 Defensive CSV ingestion	8
DE08-03 JSON and nested-record ingestion	11
DE08-04 Parquet ingestion and metadata inspection	14
DE08-05 REST API contracts, status codes and timeouts	17
DE08-06 API authentication, pagination and rate limits	20
DE08-07 Retries, backoff, jitter and transient failures	23
DE08-08 Database extraction and safe query boundaries	26
DE08-09 Historical batch landing and raw metadata	29
DE08-10 Incremental loads and high-water marks	32
DE08-11 CDC concepts and change-event semantics	35
DE08-12 Schema drift, compatibility, time zones and quarantine	38
DE08-13 Checkpoint/state management and replay safety	41
DE08-14 Idempotent landing and content identity	43
DE08-15 Late-arriving data, event time and processing time	46
DE08-16 Ingestion observability: counts, latency and run metadata	49
DE08-17 Ingestion integration, reconciliation and backfill proof	52

START HERE

How to use this module

Open the matching practice folder from the course root: 06_M08/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

Preserve raw evidence, ingest files/APIs/databases safely, control retries and state, quarantine incompatible data, and prove replay/backfill without silent loss or duplication.

Practice: 06_M08/practice/ Quick revision:

12_REVISION_AND_TESTS/M08/M08_Quick_Revision_Book.pdf Protected answers (open only after saved attempt): 12_REVISION_AND_TESTS/M08/PROTECTED_OPEN_AFTER_ATTEMPT/M08_Module_Revision_Test_Answers.pdf

```
SOURCE -> immutable LANDING -> VALIDATE/INTERPRET -> STAGING
                                         \-> QUARANTINE
SUCCESSFUL COMPLETE RUN -----> CHECKPOINT/STATE
```

M08 is the point where a data pipeline first touches systems it does not control. A file can arrive with a changed header. An API can return JSON and still be an HTTP error. Two records can share the same timestamp. A retry can save a transient request or duplicate a non-idempotent effect. A source can change shape after your code was deployed. The lesson is therefore not "how to read a CSV". The lesson is how to ingest evidence without losing your ability to prove what arrived, what was accepted, what was rejected, where progress stopped, and whether a replay is safe.

The whole module uses one small ingestion project. Its mental model is SOURCE -> immutable LANDING -> contract validation -> STAGING or QUARANTINE -> STATE/CHECKPOINT. Landing is evidence, not a cleaned table. Staging is an interpretation under a known contract. Quarantine is not a rubbish bin; it is an explicit holding area with reasons. State is not advanced because a script printed SUCCESS; it advances only after the run's required outputs and controls are complete.

The project deliberately stays small enough to inspect by hand. A local mock API contains ten v1 orders. Two orders share the same updated_at so a timestamp-only watermark can be proven unsafe. A v2 endpoint deliberately changes customer and price fields so schema drift is visible. CSV, nested JSON, SQLite, CDC-style JSONL and late-event fixtures let you compare source contracts without needing a cloud account. Parquet code is included, but native PyArrow execution is not certified by this review release. The script is syntax-checked; collect the real Parquet runtime evidence on your learner PC after installing the supplied requirements.

The practice/ folder is a miniature ingestion platform, not a pile of disconnected exercises. source/files/ contains stable file fixtures; source/events/ contains CDC-style and late-event records; mock_ingestion_api.py is a local HTTP source; src/ contains small reference components; output/ is disposable learner output; assessments/ contains the closed-book module and fresh

changed checkpoints. Keep source fixtures unchanged so expected controls remain trustworthy.

Create a virtual environment in your own working copy and install requirements.txt. The API and all non-Parquet Python labs run with ordinary Python plus Requests. Parquet additionally needs PyArrow. Start the mock API in a separate terminal with `python mock_ingestion_api.py`; it listens only on localhost port 8008 and uses the classroom token `stage08-demo-token`. In normal study, set the token in `STAGE08_API_TOKEN` rather than editing it into client code. The token is intentionally fake, but the habit is real.

Do not write answers into the master package. Copy the practice project into your module-work folder. A failed first attempt is evidence; keep it. If review reveals the problem, add `repair_note.md`, close review, then create a separate changed retry.

1. What is the source contract and grain? A file row, API item, database row and CDC event do not automatically mean the same thing.
2. What raw evidence did I preserve? Filename alone is weak; bytes/pages, run identity and checksums/metadata are stronger.
4. What state says the run may resume here? State must use the same deterministic ordering as extraction and must not jump ahead of durable output.
5. Can I replay/backfill without losing or multiplying the intended logical result? If not, the pipeline is still a demo rather than a reliable ingest.

From the `practice/` folder, `python -m src.csv_ingest source/files/orders_2026-08-01.csv`

`output/csv` runs the file ingester. `python mock_ingestion_api.py` starts the HTTP fixture. In another terminal, `python -m src.api_client` exercises pagination/retry using the local token default. `python -m`

`src.db_extract`, `python -m src.cdc_apply`, and `python -m src.late_events` run focused labs. Treat

commands as reproducible entry points; the learning goal is the contract/evidence they expose, not typing them from memory.

DE08-01

Ingestion architecture: source -> landing -> staging

Watch first

[1. DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt](#)

OPTIONAL visual support - no independently verified bounded range. Use only for architecture/ingestion context; learner book + official architecture reference + lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: M07 complete. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- VISUAL: DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt - OPTIONAL visual support
- - no independently verified bounded range. Do not watch the whole recording for this lesson. Use it only to
- visualize ingestion architecture; the learner book + official architecture reference + lab are the required
- route.
- OFFICIAL: Microsoft - Data lake architecture - Use raw/cleansed/curated separation to connect source
- preservation with replay/auditability.

Why ingestion needs layers before it needs tools

Imagine a courier delivers a sealed envelope. If you immediately open it, rewrite the address, throw away the envelope and only keep your cleaned note, you can no longer prove what was actually delivered. Ingestion has the same problem. Source is what the producer exposes. Landing is your preserved copy of what you received. Staging is the version you have interpreted under a contract. Quarantine holds records or batches that cannot safely satisfy that contract. State says how far a repeatable process has progressed.

The most important boundary is landing versus staging. Landing should be append-only or immutable for a run: original bytes/page payload, retrieval time, source identity, checksum and other acquisition metadata. Staging can be rebuilt. If a parsing rule was wrong, you should be able to delete staging, fix the parser, and reconstruct from landing without asking the producer to resend yesterday's data.

Follow one order through the system

Suppose the API returns order A4007 at 2026-08-05T09:00:00Z. The landing layer stores the exact response page plus a manifest. Validation checks required keys and types. A valid record

becomes a staging row. If v2 suddenly sends price instead of unit_price, the raw v2 response still belongs in landing, but the record should go to quarantine until an explicit compatibility mapping exists. Preserving the raw evidence and refusing unsafe staging are both success behaviours.

A useful run folder is intentionally boring:

```
landing/source=orders_api/run_id=20260914T100501Z/  
  page_001.json  
  page_002.json  
  _manifest.json  
staging/orders.csv  
quarantine/orders.jsonl  
state/orders_checkpoint.json
```

The names make ownership visible. The checkpoint is not hidden inside a log line, and quarantine is not mixed with accepted rows.

The dangerous shortcut

A beginner often writes `requests.get(...).json()` and directly appends the parsed records to the final table. That collapses acquisition, validation and transformation into one irreversible step. If page 3 fails after pages 1 and 2 were already written, what does the checkpoint mean? If tomorrow's schema changes, which historical payload produced a suspicious row? If a bug in type conversion is discovered, can the original bytes be replayed? Layering is the answer to those recovery questions.

Independent proof

Draw the five-part flow from memory and annotate where raw bytes, accepted rows, rejected rows and progress state live. Then take `orders_2026-08-01.csv` and design the exact landing path/manifest fields

you would save before parsing. State whether landing may be overwritten on retry and why.

Validate

Your design passes only if: raw evidence survives parser changes; staging can be rebuilt; rejects retain source identity/reason; state advances after completed success; a second run cannot silently overwrite the first run's evidence.

Changed retry

A vendor sends the same filename `orders.csv` every day. Design a landing convention that proves which bytes belonged to Monday and Tuesday even though the source filename is identical. Explain why renaming the newest file in place is weaker than a run-scoped immutable landing copy.

Break the architecture on purpose

Try the shortcut design source -> `staging.csv`, where every run overwrites the same file. Now ask four recovery questions. If a parser bug is found tomorrow, where are yesterday's original bytes? If the source resends a corrected file under the same name, can you prove which version created the current staging rows? If the process stops halfway through, can you distinguish a partial staging file from a complete one? If a schema change causes rejection, do you still possess the raw payload needed to write a new mapping? The shortcut fails all four questions.

A second failure is to call quarantine "failed rows" and then delete it after a successful run. A quarantined row is part of the source accounting. If `source_rows=100` and `staged=97`, the other three must be explainable. Keeping their original source position and reason turns 97 into a defensible result; deleting them turns 97 into unexplained loss.

What a strong engineer can say

"Landing proves what arrived. Staging proves what my current contract accepted. Quarantine proves what it rejected and why. State proves how far the repeatable process is allowed to resume. I can rebuild staging from landing without changing raw evidence, and I do not move state merely because acquisition succeeded."

Recovery ladder

If the layers feel abstract, take one physical source file and write four filenames on paper: raw copy, staged rows, rejected rows, checkpoint. Ask what is allowed to change after a parser bug. Raw copy should not; staged/rejected output may; checkpoint changes only after the intended successful boundary. Then redraw the architecture.

Before you move on

Teach it back: Explain why a successful parse can still be an ingestion failure. Use the words landing, staging, quarantine and state, and give one example where raw evidence lets you repair yesterday without asking the source to resend data.

Minimum saved evidence: architecture sketch + one proposed landing path/manifest + state-commit sentence. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-02

Defensive CSV ingestion

Watch first

[1. DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt](#)

OPTIONAL visual support - no independently verified bounded range. Use only for file/CSV ingestion context; Python csv docs + learner book + lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-01 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- OFFICIAL: Python csv module documentation - CSV is a serialization format with dialect/quoting rules;
- course lab adds contracts, checksum and quarantine.
- VISUAL: DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt - OPTIONAL visual support
- - no independently verified bounded range. Do not watch the whole recording for this lesson. The Python
- csv documentation + learner book + defensive CSV lab are the required route.

CSV is a byte format, not a trustworthy table

CSV looks simple because you can open it in a spreadsheet. But commas can appear inside quoted text, encodings can differ, headers can drift, blank strings can mean different things, and a value that looks numeric may be an identifier that must retain leading zeros. Treat every CSV as an external contract.

The lab's good file contains four order rows. The bad file deliberately includes defects. The job is not to make the bad file disappear. Land its bytes, identify exactly which rows fail, quarantine them with reasons, and keep enough evidence to reproduce the decision.

Start with acquisition evidence

Before parsing, calculate SHA-256 and byte length:

```
raw = path.read_bytes()
sha = hashlib.sha256(raw).hexdigest()
```

A checksum is not a data-quality score. It is an identity check: "these are the same bytes I received earlier." If the checksum changes, you have a different content version even when the filename is identical.

Parse with a declared contract

For this project the staging contract requires: order_id, updated_at, customer_id, product_id, qty, unit_price, currency. order_id is text. qty is a positive integer. unit_price is a non-negative decimal-like number. updated_at must represent an explicit UTC instant ending in Z. Required identifiers cannot be blank.

A useful validation order is: header -> per-field parse -> record-level rules -> duplicate/business-key rules -> batch-level controls. Do not convert first and ask questions later. If qty='five', keep the original raw value in the rejection evidence rather than replacing it with NULL and pretending the record is merely incomplete.

Quarantine is evidence, not silent dropping

A rejected record should carry at least source file, source row number, original values and one or more reasons. Example:

```
{"source_row": 4, "order_id": "B1004", "reason": "qty_not_positive", "raw_qty": "0"}
```

If a row fails two rules, decide whether to record all detected reasons or the first blocking reason; document the policy. What you must not do is continue with no evidence.

Reconcile the batch

If the parser reads 6 data rows, then: accepted + quarantined = source_data_rows. That equation catches silent loss. Separately assert accepted business keys are unique if the contract requires uniqueness. A row-count equation cannot prove values are right; a unique-key check cannot prove no row vanished. Use both.

Independent proof

Run the reference CSV ingester on the good and bad fixtures, then build your own small version under practice/independent_DE08_02.py. Before opening review, save source row count, accepted count, quarantine count, SHA-256 and at least one rejected-row reason.

Changed retry

Create a new CSV with a quoted comma in a product description, one duplicate order_id, and one leading-zero customer ID. Predict what a naïve line.split(',') parser would do. Then use the CSV parser correctly and prove the identifier's leading zeros survive.

Break the CSV parser on purpose

Open orders_bad.csv and predict each failure before execution. X1002 has quantity zero, which is syntactically numeric but violates the positive-quantity rule. The first X1003 uses an offset-less timestamp and a word where an integer is required. The second X1003 repeats a business key, has a missing customer and a negative price. One row can therefore have several independent reasons.

Now imagine code that starts with `df = pd.read_csv(...); df['qty']=pd.to_numeric(df.qty,`

`errors='coerce');` `df=df.dropna()`. It looks efficient but it converts a parse problem to NaN and then silently removes evidence. The final row count may look clean while you have no record of what disappeared. A defensive ingest separates parsing, validation and disposition.

Exact expected evidence

For the supplied good file, source=4, accepted=4, quarantined=0. For the supplied bad file, source=4 and accepted+quarantined must still equal 4. The package QA currently accepts only X1001 and quarantines three records. Your own implementation may record more than one reason per rejected row, but it must not stage invalid records just to preserve count.

Recovery ladder

If the CSV will not parse, first inspect raw bytes/encoding and header rather than changing validation. If it parses but counts disagree, number source rows before filtering. If leading zeros disappear, inspect the dtype/serialization boundary. If duplicate handling is unclear, state whether uniqueness is per file, per run, or global business key before writing code.

Before you move on

Teach it back: Explain why accepted + quarantined = source rows is useful but not sufficient by itself. Add one independent key/value control and one example of a value that must remain text even though it looks numeric.

Minimum saved evidence: good/bad run outputs + SHA-256 + accepted/quarantine reconciliation + rejection example. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-03

JSON and nested-record ingestion

Watch first

[1. DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt](#)

OPTIONAL visual support - no independently verified bounded range. Use only for nested-record normalization context; learner book + JSON lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-02 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- OFFICIAL: Python json module documentation - Use object/array decoding behavior; the course owns
- output-grain and lineage rules.
- VISUAL: DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt - OPTIONAL visual support
- - no independently verified bounded range. Use only to visualize nested-record normalization; the Python
- json documentation + learner book + nested JSON lab are the required route.

Nested JSON forces you to choose output grain

JSON can represent objects inside objects and arrays inside objects. That means one source object does not automatically equal one staging row. The crucial question comes before `json_normalize`: what does one output row represent?

The fixture `orders_nested.json` has order J3001 with two items. At order grain it is one order. At item grain it becomes two rows. Neither is universally correct. The correct grain depends on the destination contract.

Follow the flattening decision

Source shape:

```
{
  "order_id": "J3001",
  "customer": {"id": "C301", "name": "Asha"},
  "items": [
    {"product_id": "P01", "qty": 2, "unit_price": 120.0},
    {"product_id": "P02", "qty": 1, "unit_price": 300.0}
  ]
}
```

If staging is one row per order item, each row must repeat the parent identifiers needed for traceability: `order_id`, customer ID, `updated_at`, plus a stable line identifier. A useful line key

could be (order_id, line_no) when source order is stable. If the source supplies a real item ID, prefer that business identity rather than inventing position semantics.

The multiplication trap

If you flatten items and then sum an order-level shipping fee repeated on each item row, you recreate the same grain bug you learned in SQL joins. Parent values copied for context are not automatically additive at child grain. Mark which fields are measures at the output grain and which are repeated descriptors.

Missing and null are different source facts

JSON can omit a key entirely or include "currency": null. Those conditions may have different meanings. Preserve enough source context that validation can distinguish "producer omitted field" from "producer explicitly supplied null" when the contract cares.

Arrays can be empty

An order with "items": [] produces zero item rows if you simply loop the array. Does that mean the order should vanish? At item grain, perhaps yes for the item table, while the order header belongs in a separate table. This is why one giant flattened table is often a poor design for nested business objects.

Independent proof

Flatten the fixture to one row/item. Prove J3001 creates exactly two rows and state the composite key. Then calculate item gross independently from the source structure and from flattened rows; totals must match. Save one note explaining which parent fields may be repeated safely and which must not be summed at item grain.

Changed retry

Add an order J3003 with an empty items array and a customer object missing name. Decide what belongs in an order-header table, what belongs in item staging, and what (if anything) should be quarantined. Do not solve by forcing a fake item row unless the destination contract explicitly requires one.

Work the grain by hand before coding

For J3001, order grain gives one header with two items. Item grain gives (J3001,1) and (J3001,2). J3002 gives one item row. Therefore the fixture should produce three item rows total. Compute gross two ways: from nested source arrays and from flattened rows. J3001 is $2 \times 120 + 1 \times 300 = 540$; J3002 is $4 \times 75.5 = 302$; combined item gross is 842. If flattened total differs, the bug is in expansion or type conversion.

A subtle trap appears when parent fields are repeated. Suppose the order header contains shipping_fee=20. Repeating 20 on each J3001 item is fine for context, but summing it at item grain would charge 40. The same lesson from SQL cardinality now appears during JSON normalization: repeating a descriptor is not permission to aggregate it.

Break it on purpose

Remove the second item from the array but leave a cached item_count=2 field on the parent. Which source fact wins? Do not invent an answer. Record the contract conflict and quarantine or repair according to documented source authority. In ingestion, contradictory fields are a

data-quality event, not a reason to choose whichever makes the code easiest.

Recovery ladder

If flattened rows are too many, inspect which arrays are exploded and whether two arrays created a Cartesian multiplication. If rows are missing, inspect empty arrays and missing nested keys. If keys are unstable, prefer source item IDs where available; otherwise document why positional line_no is acceptable for this fixture.

Before you move on

Teach it back: Explain the difference between source-object grain and flattened staging grain. Use J3001 to show why one source object can create two rows and why repeated parent measures can be dangerous.

Minimum saved evidence: flattened rows + grain/key sentence + source-versus-flat gross reconciliation. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-04

Parquet ingestion and metadata inspection

Watch first

[1. DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt](#)

OPTIONAL visual support - no independently verified bounded range. Use only for high-level file/Parquet context; Arrow docs + learner book + lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-03 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- OFFICIAL: Apache Arrow - Reading and Writing Parquet - Use write/read, projection and ParquetFile
- metadata sections.
- VISUAL: DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt - OPTIONAL visual support
- - no independently verified bounded range. Use only for high-level file/Parquet context; Apache Arrow
- documentation + learner book own the metadata/projection evidence.

Parquet is not "CSV but smaller"

CSV stores a stream of text records. Parquet stores typed columnar data with metadata. That physical difference matters because analytical readers can request only selected columns and can use metadata/statistics to avoid reading irrelevant row groups.

Picture a warehouse where all values for region are stored in one shelf section and all values for amount in another. A query needing only those fields need not unpack every unrelated column. Parquet's real layout is more precise than that analogy - row groups contain column chunks - but the picture explains why projection is natural.

The first proof is logical parity

Convert a small CSV to Parquet with explicit or inspected types. Then check:

- source logical row count = Parquet logical row count,
- expected column names/types exist,
- a projected read returns only requested columns,
- key values still match.

Compression ratio is interesting, but a tiny classroom file may not compress impressively. Correctness comes first.

Inspect metadata, not just a DataFrame

With PyArrow:

```
pf = pq.ParquetFile('orders.parquet')
print(pf.schema)
print(pf.metadata.num_rows)
print(pf.metadata.num_row_groups)
for i in range(pf.metadata.num_row_groups):
    rg = pf.metadata.row_group(i)
    print(i, rg.num_rows, rg.total_byte_size)
```

You can inspect column metadata/statistics where available. The lesson is not to memorize every PyArrow property; it is to know that a .parquet extension alone proves nothing about schema, row count, row groups or statistics.

Projection and filter are different

Projection means reading fewer columns. Predicate pushdown/row-group pruning is about avoiding some rows/row groups when metadata permits. Do not claim "Parquet only reads matching rows" without checking the engine and metadata. Later Spark/lakehouse modules will go deeper.

Current build limitation

This package includes `parquet_lab.py` and pinned requirements. The script is syntax-checked, but native PyArrow execution remains a learner-PC proof. Run it after installing the supplied requirements and save the actual schema, row-group/metadata and projection evidence. Do not mark the runtime step complete from static checks alone.

Independent proof

Create the course Parquet file. Record schema, row count, row-group count and a projected read of `order_id`, `updated_at`, `qty`. Reconcile projected row count to the source. Save the observations, not only a screenshot of successful execution.

Changed retry

Write a second Parquet file with a deliberately different row-group size. Compare metadata and explain what changed physically while the logical table remains the same.

Read the metadata like evidence

After writing the tiny file with `row_group_size=2`, four source rows should normally produce two row groups. Record what your actual PyArrow version reports rather than copying that expectation blindly. Inspect schema types: a customer identifier loaded incorrectly as integer may lose 0001 even though Parquet itself is perfectly capable of storing strings. The file format cannot rescue a wrong DataFrame dtype chosen before writing.

Projection proof is simple: request only `order_id`, `updated_at`, `qty`. The resulting table should contain those columns and the same four logical rows. This does not prove every engine performed predicate pushdown; it proves column projection and logical parity.

Break it on purpose

Write a second file after allowing pandas to infer `customer_id` numerically. Read it back and compare values with the CSV source. If leading zeros disappeared, the failure happened before/during conversion, not because Parquet randomly changed them. Then correct the dtype and rewrite from the original landing source.

Recovery ladder

`ModuleNotFoundError`: pyarrow means runtime dependency is missing; do not replace the lesson with fake output. A schema mismatch means inspect source dtype and Arrow schema before worrying about compression. Wrong row count means reconcile before continuing to Spark/lakehouse. Metadata without a source-row control is interesting but not sufficient.

Before you move on

Teach it back: Explain row group, column projection and logical row parity without saying "Parquet is faster because it is compressed." State what evidence you would inspect before trusting a newly produced Parquet file.

Minimum saved evidence: schema + rows + row groups + projected columns/rows; runtime version recorded. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-05

REST API contracts, status codes and timeouts

Watch first

[1. Corey Schafer - Python Requests tutorial](#)

Optional visual - no verified bounded range. Focus on HTTP requests, response status, JSON and authentication. Practise retries and pagination with the supplied local service; this recording does not replace those controls.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-04 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- OFFICIAL: Requests Quickstart - Status, JSON decoding, `raise_for_status` and timeout behavior.
- OFFICIAL: Requests API - Use request/response/exception reference as needed.

JSON body and HTTP success are separate questions

An HTTP response has a status code, headers and body. A server can return a perfectly valid JSON error document with status 401, 404 or 500. Therefore `response.json()` succeeding does not mean the request succeeded.

The local API gives four simple cases: `/health` -> 200; `/v1/orders` without a bearer token -> 401; an authenticated order request -> 200; an unknown path -> 404. Run all four and write the expected client action beside each.

Follow the safe request shape

```
r = requests.get(url, headers=headers, timeout=5)
print(r.status_code, r.headers.get('Content-Type'))
r.raise_for_status()
payload = r.json()
```

Why put `raise_for_status()` before trusting the payload? Because an error body may use a completely different schema. Your normal order parser should not try to interpret `{"error": "invalid token"}` as an order page.

Timeout is part of correctness

Requests does not give you a universal safe timeout automatically. A process that can wait forever is not operationally reliable. In a real system, connect and read timeouts may be tuned separately; at this stage, the habit is simpler: every network request has an explicit timeout and timeout failure is handled as a failure, not as an empty result.

Contract means more than field names

For /v1/orders, a useful contract includes: allowed status, JSON content, schema_version, page metadata, items array and expected record fields. If the server returns 200 but items is a string, HTTP succeeded but the data contract failed. These are different validation layers.

Independent proof

Call all four endpoints. Save status, content type/body summary, whether `raise_for_status()` raises, and the action your ingestion code should take. Then modify the client to use a very small timeout against a deliberately delayed local test or unreachable port and record the exception class. Do not catch Exception and label everything "API failed"; preserve useful failure category.

Changed retry

Imagine an endpoint returns status 200 with body `{"status":"error","items":[]}`. Decide whether HTTP success alone permits checkpoint advancement. Write the additional application-level contract you would need.

Build a response decision table

Before coding a generic handler, write what the client should do. 200 + expected contract -> continue. 200 + wrong body shape -> contract failure, do not checkpoint as clean success. 401 -> authentication/configuration failure, stop and repair credentials. 404 -> endpoint/path contract problem. 429 -> rate-limit handling belongs to DE08-06/07. 500 -> candidate transient failure under bounded retry.

A second axis is transport failure: DNS/connection/timeout may produce no HTTP response at all. Your error evidence should distinguish "server replied 500" from "no response before timeout" because diagnosis and retry policy differ.

Break it on purpose

Change the client order to `payload=r.json(); r.raise_for_status()`. The 401 JSON body decodes successfully, so naïve logging can say "parsed 1 object" before raising. This is why successful parsing is not a success metric. Then simulate a 200 body missing items; `raise_for_status()` passes, yet the application contract must fail.

Recovery ladder

If you see a JSON decoder error, first inspect status and content type/body prefix. If status is unexpected, do not repair the parser. If timeout occurs, verify endpoint/port and explicit timeout value. If the API returns 200 but contract fails, save the raw response/page and quarantine/fail the batch rather than coercing it into staging.

Before you move on

Teach it back: Explain the difference between transport success, HTTP success and application/data-contract success. Give one case where JSON parsing succeeds but the ingest must fail.

Minimum saved evidence: response decision table + four endpoint observations + timeout/error evidence. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-06

API authentication, pagination and rate limits

Watch first

[1. Corey Schafer - Python Requests tutorial](#)

Optional visual - no verified bounded range. Focus on HTTP requests, response status, JSON and authentication. Practise retries and pagination with the supplied local service; this recording does not replace those controls.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-05 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- OFFICIAL: Requests authentication documentation - Authentication mechanics; course lab uses a local
- bearer-token API.
- OFFICIAL: Requests Quickstart - Headers and request parameters; pagination/rate-limit contract is
- course-owned.

Pagination turns one logical extract into several requests

A paginated API is one dataset spread across pages. Correctness means you retrieve each eligible record exactly once according to the API's pagination contract, not that page 1 looked good.

The mock endpoint caps page size and returns next_page. With page_size=3, ten records arrive across four pages. Before coding, predict page sizes: 3, 3, 3, 1. After coding, prove ten unique order IDs and no gaps.

Credentials are configuration, not source code

The classroom token is intentionally fake, but use it like a real secret: environment variable -> Authorization header. Do not print it in logs or write it into Git-tracked configuration. .env.example may name STAGE08_API_TOKEN without containing a real credential.

Rate limiting is server feedback

The mock API can return one 429 on page 2 and a Retry-After: 1 header. That is not the same as a malformed request. A bounded client should wait according to the server instruction (within a sensible cap), retry that page, and continue without duplicating page 1.

A dangerous pagination loop resets accumulated results on each retry, or increments the page before the current page is durably accepted. Another dangerous loop stops when a page

contains fewer than the requested size even though the API contract says to follow `next_page`. Use the contract, not a guess.

Independent proof

Fetch all ten v1 records with page size 3. Trigger the one-time page-2 429. Record request sequence/statuses, final unique count and sorted IDs. Prove no bearer token appears in your saved log. Then rerun with a wrong token and prove the client does not repeatedly retry 401.

Changed retry

Set page size 4. Predict page count and sizes before execution. Then change only the pagination contract in a small mock response so the next cursor is a string token rather than a page number. Explain why production code should isolate pagination mechanics instead of scattering `page += 1` through transformation logic.

Prove completeness, not just page count

With ten records and `page_size=3`, the expected IDs are A4001 through A4010. Four successful page payloads are necessary but not sufficient: page 2 could accidentally repeat A4001-A4003. Therefore validate both count and identity. For this fixture, `len(rows)=10`, `len(set(order_id))=10`, first/last IDs and sorted sequence are easy independent controls.

When the one-time 429 occurs on page 2, the trace should show page 1 success, page 2 rate limit, page 2 success, then remaining pages. Page 1 should not be appended twice. The retry belongs to the current page request, not to restarting the entire extraction unless your design deliberately makes whole-run replay idempotent.

Break it on purpose

Move `page = page + 1` before the request succeeds. A 429 then advances the cursor and page 2 can be skipped. Alternatively, append items before verifying/recording the page identity and retry the same page without dedupe; duplicates appear. Both bugs can survive if you only check that the script ended.

Recovery ladder

Missing IDs -> inspect pagination state transitions and server `next_page`. Duplicate IDs -> inspect append timing and page retry. Token in logs -> stop and remove/redact it before sharing evidence. Infinite loop -> log page/cursor plus previous cursor and assert progress changes or terminates.

Before you move on

Teach it back: Explain how pagination state changes only after a page is accepted. Include how a 429 differs from a 401 and why a complete run needs identity checks in addition to total count.

Minimum saved evidence: page/attempt trace + ten unique IDs + rate-limit evidence + secret-leak check. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-07

Retries, backoff, jitter and transient failures

Watch first

[1. Corey Schafer - Python Requests tutorial](#)

Optional visual - no verified bounded range. Focus on HTTP requests, response status, JSON and authentication. Practise retries and pagination with the supplied local service; this recording does not replace those controls.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-06 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- OFFICIAL: urllib3 Retry reference - Retry categories/backoff concepts; course lab makes retryability
- explicit.
- OFFICIAL: Requests Quickstart - Timeout/connection/HTTP exception behavior.

Retry only what has a reasonable chance to recover

Retries can turn a brief network problem into success, but they can also multiply load, delay deterministic failures or repeat side effects. Reliability begins by classifying failure.

Useful beginner categories:

- connection/timeout: often transient;
- 429: capacity/rate feedback; respect Retry-After when present;
- selected 5xx: server-side/transient candidates;
- 400 validation error: usually deterministic until request changes;
- 401 authentication error: do not hammer the server with the same bad credential;
- unknown write outcome: risky because the server may have committed even if the client never received
- the response.

Exponential backoff with jitter

Without jitter, hundreds of clients can fail together, sleep the same amount and retry together - a thundering herd. A simple teaching formula is: $\text{base} * 2^{(\text{attempt}-1)} + \text{random_jitter}$. The exact policy belongs to the service context. The important controls are maximum attempts, maximum wait, retryable statuses/exceptions and observability of each attempt.

Run the failure simulator

fail_once=1 returns one 500 for the request key, then success. Your client should record attempt 1=500, wait, then attempt 2=200. Wrong token returns 401 every time; prove it is not placed in the retry set.

Idempotency warning

GET is normally intended to be safe/idempotent, which makes retry reasoning easier. A POST that creates a payment is different. If the connection drops after the server committed, repeating the POST can duplicate the effect unless the API supports an idempotency key or another deduplication contract. Do not copy GET retry rules blindly onto writes.

Independent proof

Implement a bounded retry wrapper for the local GET endpoint. Test one-time 500, one-time 429, invalid auth and an unreachable port. Save attempts/status/exception plus final outcome. A pass requires the invalid-auth case to fail quickly rather than burn all retries.

Changed retry

You now receive 503 three times and then 200. Maximum attempts is 3 total, not 3 retries after the first call. Predict whether success is reachable under that definition, then make the attempt-count wording explicit in code and logs.

Make attempt semantics unambiguous

Teams often say "three retries" when one person means three total calls and another means one original plus three retries. The course code uses max_attempts: the total number of calls including the first. Log attempt=1/4 style evidence so operations can interpret it.

For the one-time 500 test, the trace should contain 500 then 200. For 429, it should contain 429 then 200 while respecting Retry-After. For wrong token, the first 401 should raise without consuming all attempts. For an unreachable port, connection errors may consume the bounded attempt budget.

Break it on purpose

Add 401 to retry_status. The client now repeats the same bad authentication four times, increasing load without changing the cause. Then set no max attempt bound and imagine a permanently failing service: your pipeline can hang indefinitely. Finally retry a non-idempotent write after an unknown timeout; you may duplicate a real side effect.

Recovery ladder

Repeated deterministic status -> reclassify retryability before adjusting backoff. Excessive wait -> inspect maximum attempts/wait caps. Thundering-herd risk -> add jitter. Unknown write outcome -> do not solve with larger retry count; find the API's idempotency/deduplication semantics.

Before you move on

Teach it back: Explain which failures you would retry and which you would not. Define max_attempts clearly and explain why retrying an unknown non-idempotent write is more dangerous than retrying GET.

Minimum saved evidence: retry matrix + 500/429/401/connection traces + declared max-attempt semantics. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-08

Database extraction and safe query boundaries

Watch first

[1. Mike Bayer - SQLAlchemy 2.0 getting started \(embedded instructor video\)](#)

Optional visual - no verified bounded range. Play the first tutorial, SQLAlchemy 2.0 - The One-Point-Four-Ening. Focus on engine, connection, SELECT and transaction boundaries. The course examples use explicit transactions.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-07 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- OFFICIAL: PostgreSQL 18 - COPY - Use client/server file boundary and query-output extraction awareness.
- OFFICIAL: Python sqlite3 documentation - Course uses a local SQLite source so safe extraction can run
- without a second service.

Database extraction is a source-safety problem before it is a SQL problem

A source database is often operational. Your pipeline should extract the intended rows without accidentally mutating source state or issuing an unbounded query that hurts production.

The course uses SQLite so the exercise runs locally, but the safety habits transfer to PostgreSQL: read-only credentials/transactions where possible, explicit columns, bounded date/key ranges, deterministic ordering, parameterized filters and a development LIMIT while testing.

Composite boundary query

```
SELECT order_id, updated_at, customer_id, amount
FROM source_orders
WHERE updated_at > ?
      OR (updated_at = ? AND order_id > ?)
ORDER BY updated_at, order_id
LIMIT ?;
```

The parameters are values, not string concatenation. This makes the boundary explicit and avoids accidentally changing SQL syntax with data.

Why explicit columns matter

SELECT * is convenient in a notebook but creates an implicit contract with every current and future source column. If a producer adds a large blob or sensitive field, your extraction shape silently changes. Explicit columns document what you consume.

Read-only is layered defence

A query that contains only SELECT is one defence. A read-only connection/role is another. Least privilege protects you when code changes later. In PostgreSQL, COPY (SELECT ...) TO ... is server-side and \copy is a psql client command; understand which machine reads/writes the path.

Independent proof

Create the packaged SQLite source. Extract after watermark (2026-08-05T08:15:00Z, D5002) with a LIMIT. Save SQL text, bound parameters, returned rows and proof that the source row count/hash did not change. Then try an UPDATE on a read-only SQLite URI connection and record the failure.

Changed retry

The source gains a column internal_notes. Your extraction should remain unchanged because it names explicit fields. Prove it, then explain why schema drift in unused source columns may be non-breaking while drift in a consumed field may be breaking.

Prove the source stayed unchanged

Before extraction, record source row count and optionally file/database hash for the local fixture. After the read-only query, verify the same control. The point is not that SELECT usually mutates; it is to build the habit that source safety is observable.

The expected extraction after (08:15,D5002) is D5003 followed by D5004. D5003 shares the timestamp with D5002 but its ID is greater, so it qualifies. This is the same composite-ordering pattern later used for incremental APIs.

Break it on purpose

Use only updated_at > '08:15': D5003 disappears. Use only updated_at >= '08:15': D5002 is repeated. Use string interpolation with a value containing a quote: query syntax can be corrupted. Parameter binding and tie-breakers solve different failure classes.

Recovery ladder

Unexpected rows -> print bound marker values and ORDER BY, not only the SQL template. Missing tied row -> inspect the secondary key condition. Slow/unbounded dev query -> add deterministic LIMIT and narrow range while validating. Mutation succeeds on a supposed read-only connection -> your safety boundary is not actually read-only; fix role/URI/transaction before continuing.

Before you move on

Teach it back: Explain how read-only access, explicit columns, parameter binding, deterministic order and LIMIT protect different parts of source extraction. Show why each is not a substitute for the others.

Minimum saved evidence: SQL + parameters + returned IDs + source unchanged/read-only proof. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-09

Historical batch landing and raw metadata

Watch first

[1. Data Engineering Zoomcamp - ingestion with dlt](#)

Optional visual - no verified bounded range. Focus on source acquisition, incremental loading and replay. The supplied exercises define their own raw identity and progress contracts.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-08 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- OFFICIAL: Microsoft - Data lake architecture - Raw/bronze retention supports traceability and reprocessing.
- COURSE: Course historical landing simulator - The runnable lab is authoritative for manifest/checksum/run-id behavior.

A historical backfill starts by preserving batches, not concatenating them

Suppose finance gives you two daily CSV files six months late. The temptation is to combine them immediately into one clean file. That destroys acquisition boundaries. A better landing run records each received object independently with filename, byte size, SHA-256, retrieval time and source date/partition if known.

Manifest versus data

A manifest is metadata about landed objects. It is not a replacement for the objects. Example:

```
{
  "run_id": "20260914T120000Z",
  "files": [
    { "name": "orders_2026-08-01.csv", "bytes": 321, "sha256": "..."},
    { "name": "orders_2026-08-02.csv", "bytes": 298, "sha256": "..."}
  ]
}
```

Later, staging can also record parsed row counts, but do not mutate the original checksum to reflect transformed output.

Duplicate filename is not duplicate content

Vendors frequently resend orders.csv. Filename equality is not content identity. Conversely, the same bytes can arrive under two filenames. Decide whether your platform deduplicates by source business identity, content hash, run contract, or does not deduplicate landing at all and

deduplicates downstream. Make the policy explicit.

Partition metadata is not always event time

A file named `orders_2026-08-01.csv` might mean event date, extraction date or delivery date. Do not infer business event time from naming convention without a source contract. Preserve both acquisition metadata and source-provided event timestamps.

Independent proof

Land both historical files into one run directory without renaming their original basenames. Produce `_manifest.json` with bytes and SHA-256. Recalculate hashes independently and compare. Then rerun with a new `run_id` and prove the first landing folder remains unchanged.

Changed retry

The second file is resent with one corrected byte but the same name. Show how content hash reveals a new version. Decide whether to keep both landed copies and what lineage fields staging needs to identify which one was used.

Historical ingestion needs a receipt

Treat the manifest like a receiving log. It tells a future engineer what objects were acquired in one run before interpretation. For the two supplied files, record both original basenames, byte lengths and independent hashes. The manifest itself should have a `run_id` and acquisition time/owner/source where appropriate.

Do not rewrite the manifest entry after cleaning to contain the cleaned row count as if it described the raw file. You may add derived metadata in a separate field or downstream run record, but keep acquisition evidence distinguishable from transformation evidence.

Break it on purpose

In a disposable scratch example, compare what an unsafe overwrite would lose. The supplied landing function rejects changed bytes under the same raw identity. The latest manifest may look valid while the first acquisition disappeared. Then change to unique run IDs and prove both copies coexist. If storage cost later requires deduplication, implement a governed retention/content-addressing policy rather than accidental overwrite.

Recovery ladder

Hash mismatch -> compare exact bytes and encoding/newlines before assuming corruption. Same filename/different hash -> treat as a new content version until source semantics say otherwise. Different filename/same hash -> record that fact; do not assume two distinct business batches only from names.

Before you move on

Teach it back: Explain why `run_id` and SHA-256 answer different questions. Describe what you do when the same filename arrives with a different hash and when different filenames arrive with the same hash.

Minimum saved evidence: two raw files + manifest + independently recalculated hashes + second-run preservation proof. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture

expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

A landing name is a contract

The corrected landing function accepts a plain run ID and refuses path separators or parent traversal. It reserves `_manifest.json` for its own metadata. For the same run ID and source filename, identical bytes are a replay; different bytes raise `FileExistsError`. After that failure, inspect both the original source copy and manifest: neither may change. Use a new identity when you genuinely acquire a different source version.

DE08-10

Incremental loads and high-water marks

Watch first

[1. DataTalksClub - Docker/PostgreSQL data-engineering workshop](#)

OPTIONAL visual support - no independently verified bounded range. Use only for ingestion/database context; composite-watermark lab + database references are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-09 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- VISUAL: DataTalksClub - Docker/PostgreSQL data-engineering workshop - OPTIONAL visual support - no
- independently verified bounded range. Do not search through the full workshop for a guessed chapter. The
- composite-watermark lab + database references are the required route.
- COURSE: Composite watermark lab - The exact (updated_at, order_id) tie-breaker proof is course-owned.

Incremental ingestion is an ordering contract

A high-water mark is not "the last date I saw." It is a position in a deterministic ordering. The course API orders by (updated_at, order_id) because timestamps are not unique.

A4007 and A4008 both have 2026-08-05T09:00:00Z. If your checkpoint stores only that timestamp and the next query asks updated_at > last_ts, then whichever tied record was not processed before the checkpoint can be skipped forever. The pair solves the tie:

```
(updated_at > last_ts)
OR (updated_at = last_ts AND order_id > last_id)
ORDER BY updated_at, order_id
```

The query order and checkpoint order must match.

State advances after durable success

Imagine page 1 and page 2 are staged, page 3 fails, but you already checkpointed to page 2's last record. If your staging writes are rolled back/cleared on failure, the next run starts after rows that no longer exist. State and output commit semantics must agree.

In this classroom file pipeline, we make the rule simple: collect/validate the complete eligible run, write landing/staging evidence, then atomically replace the checkpoint file. Production systems may use database transactions, object-store commit protocols or orchestration metadata, but

the principle is the same.

The supplied reference code enforces that order explicitly. `fetch_incremental(...)` returns both rows and a candidate checkpoint but does not change the live state file. Persist and validate the rows first; only

```
then call commit_checkpoint(...):
```

```
rows, candidate = fetch_incremental(base, token, state_path)
atomic_write_json(staging_path, rows) # classroom durable-output stand-in
# validate rows / counts / keys here
commit_checkpoint(state_path, candidate)
```

If the program fails before the final line, the previous checkpoint is still authoritative. A helper that secretly advances state during fetch would violate the recovery rule even if the happy-path result looked correct.

Independent proof

Delete the normal checkpoint. Run the v1 incremental client. Verify exactly ten unique IDs and final checkpoint (2026-08-05T09:20:00Z, A4010). Inspect A4007/A4008 and explain why timestamp-only state is unsafe. Rerun normally: zero new records should be eligible.

Changed retry

Use a fresh fixture where three IDs share the same timestamp and the run intentionally stops after the second tied ID. Persist (timestamp, second_id) and prove the third ID is returned on resume. Then show what a timestamp-only watermark would do.

Walk the tied timestamp slowly

The sequence contains A4007 at 09:00 and A4008 at 09:00. Suppose a run commits state after A4007 due to a controlled batch boundary. State (09:00,A4007) means the next query can safely include A4008 with timestamp = 09:00 AND id > A4007. State 09:00 alone cannot express that position.

The final checkpoint is (09:20,A4010) only after all ten records are durably accepted for the run. A clean rerun should produce zero eligible records. That zero is not "nothing happened"; it is evidence that the watermark excludes already processed rows as designed.

Break it on purpose

Commit checkpoint after every page but write staging only at the end. Fail after page 3. The checkpoint can move past records absent from staging. Then reverse it: stage partial pages but never commit state; rerun may duplicate unless staging is idempotent. Production designs coordinate these effects with transactions/commit protocols; the classroom rule simply delays live state until complete success.

Recovery ladder

Skipped tied rows -> compare ORDER BY tuple with WHERE tuple. Duplicate rerun -> inspect inclusive/exclusive boundary and staging key. State moves on failure -> find exactly where checkpoint write occurs. State moves backward -> protect monotonic progression for the defined source ordering.

Before you move on

Teach it back: Explain the composite watermark using A4007 and A4008. State the exact bug created by `updated_at > last_ts` when the prior run stopped between tied timestamps.

Minimum saved evidence: before/after checkpoint + ten IDs + tied timestamp explanation + zero-new-row rerun. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-11

CDC concepts and change-event semantics

Watch first

[1. Binod Suman Academy - Change Data Capture \(CDC\) | Why & How | Use case | System Design](#)

OPTIONAL visual support - no independently verified bounded range. Use only for CDC event/demo context; Debezium reference + learner book + CDC lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-10 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- VISUAL: Binod Suman Academy - Change Data Capture (CDC) | Why & How | Use case | System Design -

OPTIONAL visual - no verified bounded range; CDC context only.

- OFFICIAL: Debezium Tutorials - Use snapshot/change-event behavior as a real CDC reference.

CDC records changes, not just current rows

A batch query asks "what rows match now?" Change Data Capture asks "what changes happened?" A CDC event commonly carries operation type, key, before/after values, source position and event/source timestamps. Tool formats differ, but the semantic questions are stable.

Snapshot plus change stream

Many CDC systems first take a snapshot of existing rows, then continue from a transaction-log position. The consumer must distinguish snapshot records from subsequent inserts/updates/deletes and must preserve enough source position to resume safely.

The course JSONL fixture uses events like:

```
{ "op": "c", "key": "A5001", "after": { "status": "new" }, "source_pos": 101 }
{ "op": "u", "key": "A5001", "before": { "status": "new" }, "after": { "status": "paid" }, "source_pos": 104 }
{ "op": "d", "key": "A5002", "before": { "status": "new" }, "after": null, "source_pos": 110 }
```

c/u/d are classroom operation codes. The important part is that an update is not a second independent row unless your sink model intentionally keeps event history.

Ordering and duplicates

Delivery can be at-least-once. The same source event may be delivered twice after restart. A sink needs event/source identity or idempotent upsert semantics. Business event ordering may also differ from processing arrival order; source position often matters for reconstructing database state.

Delete semantics

If you ignore deletes, your analytical copy can keep entities the source removed. Depending on the target, a delete may physically remove a row, mark it inactive, close a validity interval or append a tombstone/history event. State the policy.

Independent proof

Read `cdc_changes.jsonl`. Apply events in source position to a tiny in-memory current-state dictionary and separately preserve the event log. Prove A5001 ends in the expected final status and A5002 deletion is not silently ignored. Then duplicate one event and make the application idempotent by source position/event ID.

Changed retry

Receive an update event before its create because files arrived out of order. Decide whether to quarantine, buffer, reconstruct from snapshot or allow upsert. Explain what evidence you need rather than inventing a universal rule.

Separate event log from current-state projection

The CDC fixture contains four events but final current state contains only A5001. That is not data loss: the event log preserves all four events, while the current-state projection applies create/update/delete semantics. These are two different grains: one row/event versus one row/current entity.

A5001 is created new, then updated paid. A5002 is created and later deleted. If you simply append every after object to a table, you get duplicate A5001 versions and may never represent the delete. If you only keep current state, you lose the audit/change history. Choose the target intentionally.

Break it on purpose

Append E103 twice. Without event identity, a downstream aggregate over events can double count the update. The supplied applier deduplicates by `event_id` for the classroom scenario. In real CDC, source partition/offset/LSN or connector metadata may be a better identity; do not invent UUIDs after ingestion and expect them to deduplicate source retries.

Recovery ladder

Wrong final state -> sort/check source position and op semantics. Delete ignored -> define sink delete/tombstone policy. Duplicate effect -> identify durable source event identity. Update before create -> classify as ordering/snapshot gap, not automatically as an insert.

Before you move on

Teach it back: Explain event-log grain versus current-state grain. Use create, update and delete events to show why an append-only event table and a current entity table can legitimately have different row counts.

Minimum saved evidence: event list + final current-state map + duplicate-event test + delete-policy sentence. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-12

Schema drift, compatibility, time zones and quarantine

Watch first

[1. DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt](#)

OPTIONAL visual support - no independently verified bounded range. Use only for schema-change context; learner book + datetime/schema-drift lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-11 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- VISUAL: DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt - OPTIONAL visual support
- - no independently verified bounded range. Use only for schema-change context; the learner book +
- datetime/schema-drift lab are the required route.
- OFFICIAL: Python datetime documentation - Use aware/naive datetime definitions; course contract
- requires explicit UTC instants.

Schema drift is a compatibility decision, not an automatic rename exercise

The v1 API contract has `customer_id` and `unit_price`. The v2 endpoint deliberately sends nested `customer.id`, `price` and `source_schema_version=2`. A parser that silently says "close enough" can corrupt the stable staging contract.

Classify changes

An added optional field may be backward-compatible for a consumer that ignores unknown fields. Removing a required field is usually breaking. Renaming `unit_price` to `price` may be compatible only after an explicit mapping is reviewed. Changing a type can be subtle: numeric 12.50 becoming string "12.50" may parse, but that does not mean every string is valid numeric data.

Quarantine the unknown contract

The course v2 payload should land successfully because acquisition worked. It should not advance the normal v1 checkpoint or enter v1 staging until compatibility logic exists. Quarantine evidence should say `unsupported_schema_version=2` or name the specific missing/renamed fields.

Time zones are part of schema meaning

2026-08-05T09:00:00Z means an instant in UTC. 2026-08-05T09:00:00 has no offset. Do not silently interpret an offset-less timestamp as your laptop's local timezone. Either the source contract defines its timezone or the record is ambiguous and must be repaired/quarantined.

An aware datetime and a naive datetime are different concepts. Normalize instants to UTC for transport/state comparison while preserving source value/timezone metadata when audit needs it.

Independent proof

Call `/v2/orders` with `--no-commit-state`. Land the raw page, quarantine every incompatible record, prove zero v2 rows enter v1 staging, and prove the normal v1 checkpoint is unchanged. Then inspect a timestamp fixture containing Z, +05:30 and no offset. Convert the first two to UTC and reject/flag the third unless a source timezone is explicitly supplied.

A string merely ending in Z is not automatically a timestamp. The reference validator now parses the ISO-8601 value after confirming the explicit UTC suffix, so impossible or malformed values such as `not-a-dateZ` are rejected rather than accepted by a suffix-only check.

Changed retry

A v3 payload adds `discount_pct` but keeps all required v1 fields identical. Decide whether this is compatible for the v1 consumer and what metadata you would record so the ignored new field is not

invisible to governance.

Compatibility belongs to the consumer contract

A producer can truthfully announce "schema v2" while your v1 consumer still cannot safely use it. Compatibility is therefore relative to a consumer and its expectations. The lesson's safest default is: land v2, classify as unsupported for v1 staging, quarantine with reason, and do not advance v1 state.

Time meaning is part of schema even when the JSON type is still "string". 09:00Z, 14:30+05:30 and 09:00 are all strings, but only the first two identify an instant without extra assumptions. Schema validation that checks only Python type misses semantic drift.

Break it on purpose

Write `unit_price = row.get('unit_price', row.get('price'))` and `customer_id =`

`row.get('customer_id')` or `row['customer']['id']` with no schema-version check. Your parser now silently accepts two contracts. That may be a valid explicit compatibility adapter later, but without version tests, audit metadata and regression controls it hides a breaking source change.

Recovery ladder

Unexpected field -> compare schema version/required set before modifying parser. Offset-less time -> obtain source timezone contract; do not use local machine timezone silently. Many quarantined rows after deployment -> treat as source-contract incident, not a reason to disable validation wholesale.

Before you move on

Teach it back: Explain why schema compatibility is relative to the consumer. Compare an added optional field, a renamed required field and an offset-less timestamp, and state the safe action for the v1 staging contract.

Minimum saved evidence: landed v2 payload + quarantine reasons + unchanged v1 checkpoint + timezone classification. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-13

Checkpoint/state management and replay safety

Watch first

[1. Data Engineering Zoomcamp - ingestion with dlt](#)

Optional visual - no verified bounded range. Focus on source acquisition, incremental loading and replay. The supplied exercises define their own raw identity and progress contracts.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-12 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- COURSE: Course replay/checkpoint lab - State is written atomically only after the complete run; replay
- has explicit state policy.
- OFFICIAL: Python `os.replace` documentation - Used for local atomic checkpoint replacement.

Checkpoint files are tiny but operationally critical

A checkpoint says what work is complete enough to skip next time. If it is written too early, you skip missing output. If it is corrupted halfway through a write, the next run may not know where to resume. Treat state as a controlled artifact.

Atomic local-file update

A common teaching pattern is write-to-temp then replace:

```
tmp = checkpoint.with_suffix('.tmp')
tmp.write_text(json.dumps(new_state))
os.replace(tmp, checkpoint)
```

`os.replace` gives atomic replacement semantics on the same filesystem under normal local conditions. This does not magically coordinate multiple distributed writers; it is the correct scope for this local lab.

Replay and backfill need explicit state policy

A normal incremental run reads current state and commits new state after success. A replay may intentionally re-read old raw/pages to reproduce staging but should not necessarily advance the live checkpoint. A backfill reads a historical range with its own run identity/state policy. Mixing these modes is how old data can move a live checkpoint backward or forward unexpectedly.

Preserve first evidence

Do not delete checkpoint history just to get a clean demo. Copy it or start a named replay state. Operational debugging needs the before/after state and run evidence.

Independent proof

Run v1 normally and save checkpoint. Then replay with `--ignore-checkpoint --no-commit-state`. Verify a new landing run is produced, staging remains idempotent, and live checkpoint bytes/hash do not change. Simulate failure before commit and prove state remains at the prior position.

Changed retry

Create two independent ingestion jobs that accidentally share one checkpoint path. Explain the race/ownership problem and redesign state paths so each source/contract has a clear owner.

State is a commit record, not a scratch note

Take a hash of the live checkpoint before replay. Run replay with no-commit state. Hash it again. Equality is a simple proof that replay did not alter live resume position. The replay may still create a new landing run; those are separate concerns.

Atomic replacement protects against a process reading a half-written JSON file, but it does not prevent two writers racing to replace the same path. Ownership/locking/orchestration is a higher-level concern. This

distinction prevents overclaiming what `os.replace` guarantees.

Break it on purpose

Write directly to checkpoint JSON and kill the process after `{ "updated_at":`. The next reader gets corrupted JSON. Temp+replace avoids exposing the partial file. Then launch two conceptual jobs using one state path: each can validly write a complete file and still overwrite the other's progress. Atomic does not mean coordinated.

Recovery ladder

Corrupt JSON -> restore known prior state or deliberately rebuild from evidence; do not guess a marker. Replay moved checkpoint -> check mode flags and single commit location. Concurrent ownership -> separate state by source/job or introduce a real coordination mechanism.

Before you move on

Teach it back: Explain what atomic checkpoint replacement protects and what it does not protect. Distinguish partial-file corruption from two valid writers racing on one state path.

Minimum saved evidence: checkpoint hash before/after replay + no-commit evidence + simulated pre-commit failure result. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-14

Idempotent landing and content identity

Watch first

[1. PyCon US 2025 - Building Resilient Data Pipelines: The Power of Idempotency](#)

OPTIONAL visual support - no independently verified bounded range. Use only for idempotency context; learner book + content-identity/replay lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-13 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

OPTIONAL visual - no verified bounded range; idempotency context only.

- VISUAL: PyCon US 2025 - Building Resilient Data Pipelines: The Power of Idempotency -
- OFFICIAL: Python hashlib documentation - SHA-256 is used as evidence identity in the course landing
- manifest.

Idempotency means repeating the same logical work does not multiply the intended effect

Retries and replays are normal. Therefore ingestion should have a deliberate identity rule. "Do not run twice" is not a reliability strategy.

There are several identities in play:

- content identity: SHA-256 of exact landed bytes;
- source/business identity: order_id or event key;
- source event identity: CDC position/event ID;
- run identity: this attempt's run_id.

They solve different problems.

Same content, new run

It is perfectly reasonable for landing to keep two run folders containing the same bytes because acquisition attempts are separate evidence. Downstream staging can still be idempotent by business key/content identity. Do not confuse "immutable landing" with "duplicate final rows."

Content hash is not always a business key

Two files with different whitespace/field order can represent the same logical records but have different hashes. Conversely, two distinct business events could serialize to identical bytes in a tiny payload. SHA-256 proves byte identity, not business meaning.

Upsert/deduplication needs a conflict policy

If order A4001 arrives again with the same values, ignore/update is straightforward. If it arrives with changed `unit_price`, is that a correction, a new version or an error? Idempotency alone cannot answer. You need source semantics/versioning.

Independent proof

Land the same good CSV twice under two run IDs. Prove both manifests exist and hashes match. Then stage both through an idempotent key policy and prove one logical row/order remains. Save the exact deduplication key and conflict rule.

Changed retry

Create two files with different byte hashes but the same order IDs and values because column order differs. Explain why byte-level deduplication would keep both while business-key staging can still avoid duplicate logical rows.

Use the right identity at the right layer

Suppose two landing runs contain byte-identical `orders.csv`. Their SHA-256 values match, proving content identity. Their run IDs differ, proving separate acquisition attempts. Staging keyed by `order_id` can still contain one logical row/order. All three facts can be true simultaneously.

Now change only CSV column order. Byte hashes differ, but parsed business rows may be logically identical. A content-addressed raw store would see two byte objects; a business-key staging model can recognize the same entities. This is why one universal "dedupe key" is a design smell.

Break it on purpose

Use checksum as the staging primary key. A one-byte whitespace correction creates a second logical copy. Use only business key in landing and overwrite the old file: you lose acquisition/version evidence. Identity is layer-specific.

Recovery ladder

Duplicate staging rows -> state business grain/key and conflict policy. Duplicate raw blobs -> decide whether retention policy permits content dedupe while preserving run-to-content references. Conflicting same business key -> do not silently keep first/last; classify correction/version/error based on source contract.

Before you move on

Teach it back: Explain content identity, business identity, event identity and run identity with one example each. State why no single SHA/`order_id`/`run_id` can replace all four concepts.

Minimum saved evidence: two landing run manifests + matching/different hashes as appropriate + logical staging key/count. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code,

state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-15

Late-arriving data, event time and processing time

Watch first

[1. Data Engineering Zoomcamp - ingestion with dlt](#)

Optional visual - no verified bounded range. Focus on source acquisition, incremental loading and replay. The supplied exercises define their own raw identity and progress contracts.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-14 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- OFFICIAL: Apache Beam - event time and processing time concepts - Use only conceptual
- event-time/processing-time distinction; no Beam coding required.
- COURSE: Course late-event fixture - The learner classifies a delayed event without rewriting its business
- event time.

Arrival time and business event time are not the same clock

An order can happen at 09:00, remain offline on a device, and reach your pipeline at 12:00. Event time is when the business event happened. Processing/ingestion time is when your system handled it. Late data is normal in distributed systems.

The course late-event fixture contains an event whose `event_time` belongs before the current checkpoint/window but whose `arrived_at` is later. Do not "fix" it by replacing `event_time` with arrival time; that changes the business fact.

Why this matters before streaming

Even batch pipelines group by business dates, compute SLAs and run backfills. If a file delivered on Tuesday contains a Monday transaction, partitioning only by arrival date can misstate Monday's metrics. You may need both event partition and ingestion partition/run metadata.

Define lateness relative to a rule

"Late" is not an intrinsic property. It means arrived after some expected boundary: after daily close, after watermark, after a source SLA, etc. Record the policy. A one-hour-late event can be acceptable for one dataset and critical for another.

Correction strategy

Options include updating historical partitions, emitting adjustment records, maintaining a reprocessing window, or quarantining events beyond a hard tolerance. The course does not

prescribe one universal policy; it requires that the chosen policy preserves event time and is auditable.

Independent proof

Read the late-event JSONL. Sort by event time and separately by arrival time. Identify which records change order. Classify late events under a declared 60-minute tolerance and produce a repair/backfill list without rewriting their event timestamps.

Changed retry

Change tolerance to 15 minutes. Predict which classifications change before running. Explain why a watermark/tolerance change can alter operational state and may require deliberate reprocessing rather than silently applying a new rule midstream.

Calculate lateness explicitly

For L2, event_time is 09:10Z and arrival is 10:25Z: lag = 75 minutes, therefore late under a 60-minute tolerance. L1 has 5 minutes. L3 uses +05:30; convert both event and arrival to UTC before subtraction. This is exactly why offset-aware timestamps matter.

Sort the same events two ways. Event-time order answers business chronology. Arrival-time order answers what the pipeline saw and when. If those orders differ, that is expected in distributed ingestion, not proof one timestamp is wrong.

Break it on purpose

Overwrite event_time=arrived_at to make lateness disappear. Your SLA dashboard looks cleaner, but business chronology is falsified. Or classify lateness from local-clock strings without timezone normalization; events across offsets can be misordered.

Recovery ladder

Negative/implausible lag -> inspect timezone parsing and clock/source semantics. Too many late events -> review tolerance/source SLA, not timestamps themselves. Changed tolerance -> evaluate reprocessing/state effect before applying to already processed historical partitions.

Before you move on

Teach it back: Explain event time versus arrival/processing time using L2. State why replacing event_time with arrived_at would make the pipeline look cleaner while making the business data less truthful.

Minimum saved evidence: event-time and arrival-time ordering + lag calculation + declared tolerance + repair/backfill list. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in

M09/M10/projects/interviews.

DE08-16

Ingestion observability: counts, latency and run metadata

Watch first

[1. MIT Missing Semester - Debugging and profiling, embedded lecture](#)

Optional visual - no verified bounded range. Focus on logging, timing and finding a failing operation. Relate these demonstrations to source counts, rejected rows and run metadata.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-15 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- OFFICIAL: OpenTelemetry - Logs - Use structured telemetry concept; course lab uses a small JSON run
- record.
- COURSE: Course run-metadata lab - Counts, rejected rows, duration, checkpoint and checksums are the
- required evidence.

Observability answers "what actually happened in this run?"

A pipeline log line saying "done" is weak evidence. Ingestion observability should expose enough structured metadata to compare expected and observed behavior: run_id, source, start/end/duration, files/pages, source records, accepted records, quarantined records, retries, final checkpoint, checksums and status.

Counts form equations

For a record-oriented batch: $\text{source_records} = \text{accepted} + \text{quarantined}$ (when every source record is classified exactly once). For pagination: $\text{sum}(\text{page_item_counts}) = \text{unique_records_collected}$ only when the API does not repeat records across pages; otherwise uniqueness must be measured separately. Counts are not decorative metrics; they are reconciliation controls.

Latency has layers

Acquisition duration, source-to-arrival delay and event-time lateness are different. A fast ingestion of very old data is low processing duration but high source/event latency. Name the metric before interpreting it.

Structured run record

The course writes JSON such as:

```
{
  "run_id": "...",
  "source": "orders_api_v1",
  "status": "SUCCESS",
  "source_records": 10,
  "accepted": 10,
  "quarantined": 0,
  "retry_count": 1,
  "checkpoint_after": {"updated_at": "...", "order_id": "A4010"}
}
```

Do not put secrets or full sensitive payloads into logs just because JSON makes it easy.

Independent proof

Run a successful API ingestion and a schema-drift ingestion. Produce run metadata for both. The success run must reconcile counts and checkpoint. The drift run should be observable as a controlled rejection/quarantine state, not falsely indistinguishable from a clean success.

Changed retry

A run acquires 10 records, stages 9 and has 0 quarantine. Your status says SUCCESS. Use the count equation to detect silent loss and redefine the run status/alert condition.

Turn metrics into controls

A useful run record can answer: Did acquisition complete? How many source records were seen? How many were accepted or rejected? How many retries happened? What checkpoint existed before and after? Which raw content hashes belong to this run? How long did it take?

For the simple record classification contract, assert $\text{source} = \text{accepted} + \text{quarantined}$. A run with $\text{source}=10$, $\text{accepted}=9$, $\text{quarantine}=0$ must not be green. The missing row may be a software bug even when every individual staged row looks valid.

Break it on purpose

Log only $\text{status}=\text{SUCCESS}$ and duration. Now investigate a user complaint that one order is missing: the log cannot tell you source count, accepted count, quarantine or checkpoint. Structured evidence turns debugging from guesswork into comparison.

Recovery ladder

Count mismatch -> stop state advancement and identify unclassified records. High retry count with eventual success -> still surface degradation. Secret in logs -> treat as security incident/hygiene failure and redact/rotate as appropriate. Run metadata without run_id/source -> add identity before adding more metrics.

Before you move on

Teach it back: Explain one reconciliation equation from the module and how it turns observability into a correctness control. Include one example where $\text{status}=\text{SUCCESS}$ must be rejected by the metrics.

Minimum saved evidence: two structured run records + count equation + retry/checkpoint metadata + one alert condition. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code,

state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.

DE08-17

Ingestion integration, reconciliation and backfill proof

Watch first

[1. Data Engineering Zoomcamp - ingestion with dlt](#)

Optional visual - no verified bounded range. Focus on source acquisition, incremental loading and replay. The supplied exercises define their own raw identity and progress contracts.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE08-16 complete or repaired. Use the learner project on PC; phone is fine for reading/revision.

Useful visual / source

- COURSE: Independent integration assessment - No tutorial during first attempt. Use prior sources only for
- targeted repair after evidence is saved.
- OFFICIAL: Microsoft - Data lake architecture - Reference for replayable raw-to-curated boundaries after
- first attempt.

Integration: prove the ingestion system, not one happy-path script

This lesson combines the module into one operational story. You receive historical CSV, a current paginated API, one late event, one schema-drift page and CDC-style changes. Your job is to preserve evidence, classify contracts, stage valid data, quarantine incompatible data, advance state only after the right success boundary, and demonstrate that a replay/backfill does not create duplicate logical output.

First attempt is closed-book

Before opening review, create a new run workspace. Do not reuse the reference output folder because old evidence can make a broken pipeline look correct. Save your first attempt even if it fails.

Required proof chain

1. Source inventory - list each source object/endpoint and its contract/grain.
2. Landing - raw CSV/API/event evidence with run IDs and checksums/page metadata.
3. Validation - required fields/types, schema version, timestamp meaning and key checks.
4. Staging/quarantine - accepted/rejected counts reconcile to source scope.
5. Incremental state - composite watermark is deterministic and commits after successful run.

6. Failure handling - 429/one-time 500 recover under bounded retry; 401 and incompatible schema do not loop.

7. Replay/backfill - new run evidence appears but logical staging does not duplicate and live state follows declared policy.

8. Observability - a run record lets another engineer explain what happened without reading your source code first.

Reconciliation is stronger than "all tests passed"

Manually derive at least one expected control: API v1 unique order count = 10; final checkpoint = (2026-08-05T09:20:00Z, A4010); schema v2 contributes zero rows to v1 staging until mapped; replay does not increase unique v1 order count; source record classifications reconcile.

Failure injection

Break one thing on purpose: wrong token, one-time 500, changed schema, duplicated landed content or a simulated exception before checkpoint commit. Predict the failure and state impact first. After running, compare observed evidence with prediction. Repair only that failure, then rerun.

Handoff standard

Another learner should be able to read README/run metadata, create the environment, start the local API, execute the pipeline, locate landing/staging/quarantine/checkpoint, and reproduce the controls. Hard-coded paths to your personal desktop, hidden tokens and unexplained manual cleanup fail handoff even if your machine currently works.

Fresh changed checkpoint

The package includes a different assessment fixture with IDs/timestamps changed. Complete it without reopening the first solution. Mastery means you can reconstruct the reasoning - source contract, identity, state, validation and replay - when the values change.

Score the whole chain with independent controls

A convincing capstone-style ingest does not need huge data. It needs a complete reasoning chain. Before execution, write expected controls: historical files=2; API unique v1 orders=10; final v1 checkpoint=A4010 at 09:20Z; v2 accepted-to-v1-staging=0 until mapped; replay unique v1 staging count unchanged; L2 late under 60-minute policy; CDC current state contains paid A5001 and no A5002.

After execution, compare observed values with that sheet. If one control fails, do not average it away because the other seven passed. State/checkpoint corruption, silent data loss and unsafe schema coercion are critical failures.

Break it on purpose

Choose one failure injection and predict all affected artifacts. Example: wrong token -> no valid order pages should be staged, live state should not advance, run metadata should report authentication failure, landing may contain only permitted acquisition/error evidence according to your design. If checkpoint changes anyway, the bug is state commit logic, not authentication.

Professional handoff questions

Can another engineer identify source contracts without reading every function? Can they rerun from a blank output directory? Can they locate raw evidence and checkpoint? Are secrets external? Are expected controls documented? Can they distinguish normal run from replay/backfill? If any answer is no, the pipeline is not yet reproducible even if the demo worked once.

Recovery ladder

Start at the earliest boundary whose independent control differs: acquisition -> contract -> classification -> identity -> state -> replay -> observability. Repair the smallest boundary, preserve failed evidence, then take the changed checkpoint. Do not rewrite all code until you know which contract failed.

Before you move on

Teach it back: Give a five-minute handoff: source contracts, landing evidence, validation/quarantine, retry policy, composite state, replay rule and run controls. Then describe one failure injection and its expected artifacts before repair.

Minimum saved evidence: closed-book evidence bundle + expected-control sheet + failure injection + fresh changed checkpoint. A screenshot without the underlying values/command/path is supporting evidence, not the whole proof. If your observed value differs from the book's fixture expectation, do not edit the expectation to match your result; diagnose source version, code, state and working-directory differences first.

Review discipline and evidence

Save the first attempt before opening the matching review. Your evidence should include the relevant source identity, observed output/error, at least one independent validation control, and a short own-words explanation. After repair, close review and solve a changed retry; never overwrite the first attempt. Revision cadence: D+1, D+4, D+10, D+30, then reuse in M09/M10/projects/interviews.